

Campus Course & Records Manager

(CCRM)

Console-based Java SE application

Programming in Java - Project 1

SUBMITTED BY	Abhijit Mishra
REGISTRATION NUMBER	24BEC10033
TECHNOLOGY	Java SE 17+ / NIO.2 / Streams
REPORT DATE	18 September 2026

Table of Contents

1. Executive Summary	3
2. Requirements Traceability	4
3. System Architecture	5
4. Domain Model and Object-Oriented Design	6
5. Java Language and Core Feature Demonstration	7
6. Data, File I/O, and Reporting	8
7. Exception Handling and Validation	9
8. Build and Verification Evidence	10
9. Installation, Execution, and Eclipse Setup	11
10. Submission Deliverables and Readiness	12
11. Limitations and Future Enhancements	13
12. References	14

This report is derived from the supplied eight-page project brief and the verified contents of the CCRM Java repository. It records actual implementation evidence and explicitly identifies unfinished submission artifacts.

1. Executive Summary

Campus Course & Records Manager (CCRM) is a menu-driven Java SE application for managing students, courses, enrollment, grades, transcripts, data import/export, and backups. The project demonstrates core Java syntax and object-oriented design together with modern APIs such as Streams, NIO.2, and java.time. Its implementation is organized into seven packages and uses an in-memory singleton data store supported by CSV persistence utilities.

The current repository contains 26 Java source files and 863 source lines. A verified integration run imported the supplied CSV data, enrolled a student, recorded marks, generated a transcript with a 10.00 GPA, exported three files, and created a timestamped backup. The public GitHub repository is available at github.com/axbhijit4192-alt/CCRM-Java-Abhijit.

Project objectives

- Provide a clear console workflow for student, course, enrollment, grading, reporting, and file operations.
- Demonstrate encapsulation, inheritance, abstraction, and polymorphism in a coherent domain model.
- Apply interfaces, enums, nested classes, lambdas, Streams, recursion, assertions, and exception handling.
- Use Singleton and Builder patterns where they improve ownership and construction.
- Produce a runnable, documented repository with test data and reproducible build commands.

Project facts

Item	Verified value
Student	Abhijit Mishra
Registration number	24BEC10033
Application type	Java SE console application
Main class	edu.ccrm.cli.CCRMApp
Source scale	26 Java files; 863 source lines; 7 packages
Verified JDK	Temurin OpenJDK 21.0.6 (compatible with JDK 17+)
Repository	CCRM-Java-Abhijit / main / commit fec690e

2. Requirements Traceability

The following matrix maps the functional requirements in the supplied brief to concrete implementation evidence.

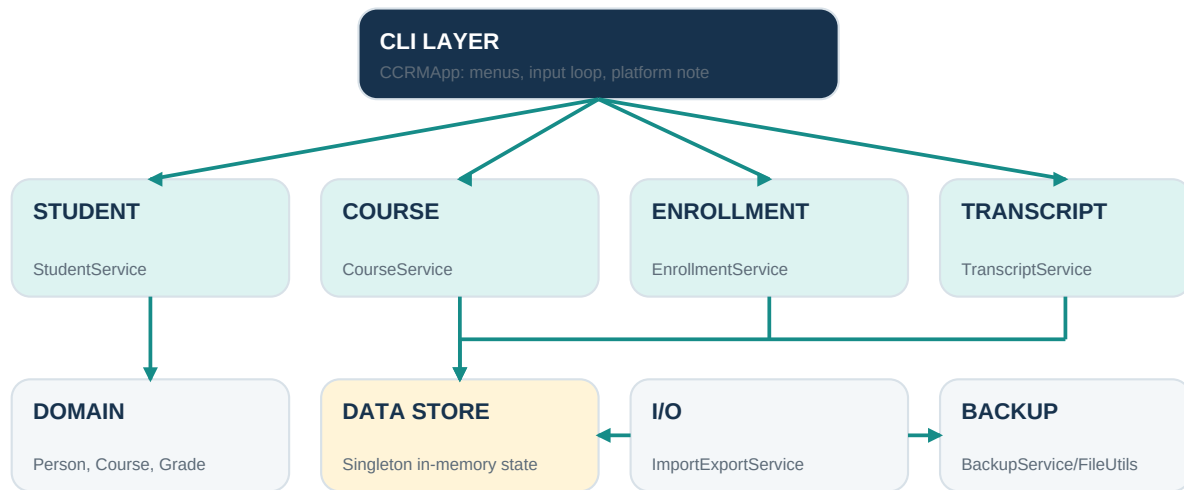
Requirement	Implementation evidence	Status
Student management	CCRMApp manages add, list, status update, and deactivate operations. Student stores registration, identity, enrollment list, status, and timestamps.	Implemented
Course management	Course.Builder creates courses; CLI supports add, list, search, update, and deactivate; CourseService filters by instructor, department, and semester.	Implemented
Enrollment and grading	EnrollmentService enrolls/unenrolls, enforces a 20-credit limit, blocks duplicates, records marks, and derives Grade values.	Implemented
Transcript and GPA	TranscriptService builds per-student transcript lines and computes credit-weighted GPA; Transcript.toString renders the result.	Implemented
File operations	ImportExportService imports students/courses and exports students/courses/enrollments; BackupService creates timestamped copies using NIO.2.	Implemented
Recursive utility	FileUtils.recursiveSize walks the backup tree and sums file sizes.	Implemented
Menu workflow	CCRMApp provides seven main options and nested workflows using enhanced switch syntax.	Implemented
Screenshots	Repository includes an explicit checklist, but genuine JDK/Eclipse/runtime screenshots have not yet been supplied.	Pending
Demo video	Optional in the brief and not included in the repository.	Optional

Technical coverage summary

Area	Evidence in repository
Packages	edu.ccrm.cli, config, domain, exception, io, service, util
Java core	Primitives, objects, arithmetic/logical/bitwise operators, strings, arrays, decision structures, loops, and labeled jumps
OOP	Abstract Person; Student and Instructor inheritance; private state; overridden behavior; base-class references
Interfaces	Persistable, Searchable, ProfileLabel, RecordLabel; explicit resolution of a default-method diamond
Nested types	Static Course.Builder; non-static Transcript.InnerLine; anonymous Consumer callback
Enums	Semester with display/order fields; Grade with grade-point values and score mapping
Patterns	AppConfig/DataStore Singleton; Course.Builder
Exceptions	Two checked custom exceptions; unchecked validation exception; multi-catch and finally
Modern APIs	Streams, lambdas, NIO.2 Path/Files, LocalDateTime, DateTimeFormatter

3. System Architecture

CCRM follows a compact layered design. The CLI owns user interaction, service classes hold application rules, domain classes model records, DataStore centralizes runtime state, and I/O utilities handle external files. This separation limits direct coupling between menus and persistence details.



Layered package design with a shared singleton DataStore

Figure 1. High-level CCRM component architecture.

Package responsibilities

Package	Primary responsibility	Representative classes
edu.ccrm.cli	Menu rendering, input handling, workflow coordination	CCRMApp
edu.ccrm.config	Application configuration and data directory	AppConfig
edu.ccrm.domain	Entities, values, enums, interfaces, transcript model	Person, Student, Course, Enrollment, Grade
edu.ccrm.exception	Checked business-rule failures	DuplicateEnrollmentException, MaxCreditLimitExceededException
edu.ccrm.io	CSV import/export and backup orchestration	ImportExportService, BackupService
edu.ccrm.service	Business logic, queries, shared state, reports	DataStore, EnrollmentService, TranscriptService
edu.ccrm.util	Arrays and recursive file utilities	ArrayDemos, FileUtils

Runtime state

DataStore is a singleton with concurrent maps for students and courses plus a synchronized enrollment list. Services obtain the same instance, so imported records immediately become available to enrollment, transcript, export, and reporting workflows. Persistent storage is explicit: data remains in memory until the user exports it.

4. Domain Model and Object-Oriented Design

Core relationships

Type	Role	Important relationship
Person	Abstract base class for identity and lifecycle fields	Extended by Student and Instructor; implements Persistable
Student	Learner record with active state and enrolled course codes	Inherits Person; resolves ProfileLabel/RecordLabel default method
Instructor	Instructor identity with department	Inherits Person and overrides summary
Course	Course metadata and activity state	Created with Course.Builder; owns immutable CourseCode
Enrollment	Student-course-semester association	Stores marks, derived Grade, and enrolledAt timestamp
Transcript	Credit-weighted result view	Owns non-static InnerLine objects and computes GPA
Grade / Semester	Controlled value sets	Enums with constructors, fields, and behavior

The four OOP pillars

Pillar	How CCRM demonstrates it
Encapsulation	Private fields are accessed through methods. Domain types control state changes such as setMarks, enroll, unenroll, and setActive.
Inheritance	Student and Instructor reuse identity fields and constructor behavior from Person via super.
Abstraction	Person requires summary(); Searchable captures a reusable query capability; services hide DataStore operations.
Polymorphism	Person references point to Student instances; overridden summary/toString behavior is selected at runtime.

Default-method diamond resolution

ProfileLabel and RecordLabel both provide label() as a default method. Student implements both interfaces and explicitly overrides label(), removing ambiguity and returning the polymorphic summary. This is a concrete Java interface diamond-resolution example.

```
public class Student extends Person
    implements ProfileLabel, RecordLabel {
    @Override
    public String label() {
        return summary();
    }
}
```

Design patterns

Pattern	Implementation	Benefit
Singleton	AppConfig and DataStore	One configuration/data owner shared across workflows
Builder	Course.Builder	Readable staged construction for six course properties

5. Java Language and Core Feature Demonstration

Feature	Location and purpose
Main method	CCRMApp.main initializes a bitwise permission example and starts the application.
Decision structures	if/else checks validate filters and scores; enhanced switch dispatches CLI actions.
Loops	Main while loop, student do-while menu, for and enhanced-for iterations, break/continue, labeled break.
Arrays	ArrayDemos sorts course codes and performs Arrays.binarySearch.
Strings	CSV split, export join, equals/equalIgnoreCase, compareTo, formatting, and text blocks.
Overloading	StudentService.add accepts either a Student or individual fields.
Overriding	summary, toString, compareTo, default-interface methods, file visitor methods.
Upcast/downcast	Student collections are traversed through Person references and checked with instanceof.
Functional style	Predicates and lambdas filter courses/students and aggregate GPA buckets.

Labeled CLI loop

```
mainLoop:
while (true) {
    String choice = sc.nextLine().trim();
    switch (choice) {
        case "7" -> { break mainLoop; }
        default -> { continue mainLoop; }
    }
}
```

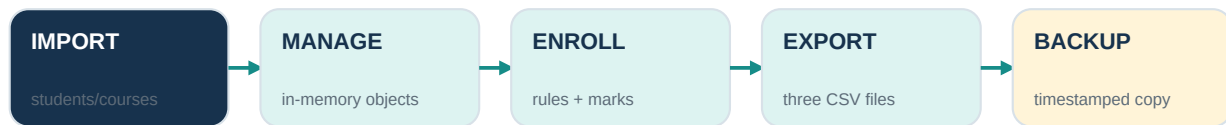
Grade enum behavior

```
public enum Grade {
    S(10.0), A(9.0), B(8.0), C(7.0),
    D(6.0), E(5.0), F(0.0);

    public static Grade fromScore(double score) {
        if (score >= 90) return S;
        // remaining bands map to A through F
    }
}
```

Assertions protect constructor invariants such as nonblank course codes and credit ranges. They are enabled at runtime with the `-ea` flag, making the distinction between development-time invariants and recoverable business exceptions explicit.

6. Data, File I/O, and Reporting



NIO.2 + Streams data lifecycle

Figure 2. Data lifecycle from CSV import to timestamped backup.

CSV inputs

id	regNo	fullName	status
1	24BEC10033	Ada Lovelace	ACTIVE
2	24BEC10033	Alan Turing	ACTIVE
3	24BEC10033	Grace Hopper	INACTIVE

The registration number was customized to 24BEC10033 in the supplied sample records. For production data, registration numbers should be unique and validated before insertion.

NIO.2 usage

Operation	Implementation
Read	Files.lines reads UTF-8 CSV streams and skips headers.
Write	Files.newBufferedWriter produces export files.
Check/create	Files.exists and Files.createDirectories prepare data paths.
Walk/copy	Files.walk and Files.copy recursively create a timestamped backup.
Move/delete	FileUtils wraps Files.move and Files.deleteIfExists.
Recursive size	Files.walkFileTree visits each file and sums attributes.size().

Stream-based reports

CourseService creates instructor, department, and semester filters with stream predicates. TranscriptService groups students into GPA ranges with Collectors.groupingBy and Collectors.counting. This separates declarative aggregation from CLI presentation.

7. Exception Handling and Validation

CCRM distinguishes JVM-level Errors from application Exceptions. Errors normally indicate unrecoverable runtime conditions; exceptions represent input, business, or I/O failures that application code can report or recover from.

Mechanism	Use in CCRM
Checked exception	DuplicateEnrollmentException and MaxCreditLimitExceededException force callers to handle enrollment rule failures.
Unchecked exception	IllegalArgumentException reports an unknown student when building a transcript; RuntimeException wraps callback/file traversal failures.
Multi-catch	The enrollment CLI catches both custom enrollment exceptions in one handler.
finally	The backup workflow prints completion regardless of success or failure.
Assertions	Course credits and course-code invariants are checked when assertions are enabled.
I/O propagation	Import/export/backup methods declare IOException and the CLI converts it to a concise user-facing message.

Business rules

- A student cannot enroll in the same course and semester twice.
- Semester credits cannot exceed the configured maximum of 20.
- Marks are converted to Grade enum values and grade points.
- A transcript requires a known student and includes only graded enrollments.
- Backups require an existing export directory.

Validation opportunities

The current CLI assumes several inputs are correctly formatted. A production revision should validate email syntax, mark ranges, numeric course credits, enum input, referenced student/course existence, CSV quoting, and unique registration numbers before modifying state.

8. Build and Verification Evidence

The repository was compiled from source with Temurin OpenJDK 21.0.6. Assertions were enabled during execution. The following integration path was executed against the committed code and supplied test data.

Build commands

```
$sources = Get-ChildItem -Recurse -Filter *.java src |  
    ForEach-Object FullName  
javac -d build $sources  
java -ea -cp build edu.ccrm.cli.CCRMApp
```

Step	Action	Observed result
1	Import students.csv	Students imported
2	Import courses.csv	Courses imported
3	Enroll student 1 in CS101 / SPRING	Enrollment accepted
4	Record marks = 95	Grade S assigned
5	Print transcript	Registration 24BEC10033; GPA 10.00
6	Export all	Students, courses, enrollments CSV files written
7	Run backup	Timestamped backup created; recursive size 424 bytes
8	Exit	Application ended normally

Observed transcript

```
Transcript for 24BEC10033  
CS101 (3 cr): S  
GPA: 10.00
```

Generated file evidence

File	Size
data/export/students_export.csv	182 bytes
data/export/courses_export.csv	178 bytes
data/export/enrollments_export.csv	64 bytes
Timestamped backup copy	424 bytes total

VERIFICATION RESULT: PASS - compilation, import, enrollment, grading, transcript, export, backup, and clean exit were observed.

Testing is currently integration-oriented and manual. A JUnit suite is not included; adding repeatable unit tests for grade boundaries, duplicate enrollment, credit limits, CSV parsing, and GPA calculations is recommended.

9. Installation, Execution, and Eclipse Setup

Windows setup

- 1. Install a JDK 17 or newer distribution, such as Eclipse Temurin.
- 2. Set JAVA_HOME to the JDK installation directory and add its bin directory to PATH.
- 3. Verify with `java -version` and `javac -version`.
- 4. Open PowerShell in the CCRM-Java project directory.
- 5. Compile and run with the commands shown in Section 8; retain `-ea` to enable assertions.

Eclipse setup

- 1. Create a new Java project and select an installed JDK 17+.
- 2. Use the repository `src` directory as the source folder.
- 3. Confirm packages appear under `edu.ccrm.*`.
- 4. Create a Java Application run configuration with main class `edu.ccrm.cli.CCRMApp`.
- 5. Add `-ea` under VM arguments, set the working directory to the project root, and run.

Minimum demonstration flow

Sequence	Demonstration
Startup	AppConfig creates/loads the data directory and CCRM prints its Java platform note.
Data	Import the two test-data CSV files and list records.
Rules	Enroll a student, attempt a duplicate if desired, and record marks.
Result	Print the transcript and GPA distribution.
Persistence	Export all records, run backup, and display recursive backup size.
Shutdown	Exit through menu option 7.

10. Submission Deliverables and Readiness

Deliverable	Current state	Notes
Public Git repository	Ready	CCRM-Java-Abhijit on GitHub; main branch at fec690e
Runnable main class	Ready	edu.ccrm.cli.CCRMAApp compiled and executed
README.md	Ready	Overview, setup, Java history/platforms, architecture, mapping, acknowledgements
USAGE.md	Ready	Short compile/run and demo guidance
Test data	Ready	students.csv and courses.csv
Project report	Ready	This PDF report
JDK verification screenshot	Pending	Must be captured from the student's own machine
Eclipse setup/run screenshots	Pending	Must be captured from the student's own Eclipse workspace
Program/export screenshots	Pending	Capture menus, operations, and export/backup folders
Demo video	Optional	Two to five minute walkthrough if submitted

Important submission note: the supplied brief states that LLM use may affect academic evaluation. Review and understand all code and report content, follow institutional rules, cite assistance where required, and do not claim unperformed work. Genuine screenshots are intentionally not fabricated in this report.

Repository access

Resource	Location
Public repository	https://github.com/axbhijit4192-alt/CCRM-Java-Abhijit
Default branch	main
Verified commit	fec690e - Customize CCRM project and align submission requirements
Local project	CCRM-Java

11. Limitations and Future Enhancements

Current limitation	Recommended enhancement
In-memory state is lost unless exported	Load the most recent export automatically or use a database.
Simple comma splitting does not support quoted commas	Adopt a robust CSV parser and validate column counts/types.
Input validation is minimal	Add reusable validators, retry loops, range checks, and clear field-level messages.
Sample registration values are duplicated	Enforce uniqueness in StudentService and use distinct test data.
No automated test suite	Add JUnit tests and a CI workflow for compilation and tests.
Single-process console UX	Add command parsing, persistence configuration, and optionally a GUI or web interface.
Export is not transactional	Write to temporary files and atomically move completed exports.
No authentication/roles	Add administrative and read-only roles if deployed beyond a classroom demo.

Suggested development roadmap

Phase	Focus	Outcome
1	Validation and tests	Safer input paths and repeatable verification
2	Persistence	Automatic load/save with durable storage
3	Reporting	Rankings, semester summaries, and exportable transcript views
4	User experience	Improved navigation, help text, and optional graphical interface

Conclusion

CCRM meets the central objective of the project brief: it is a runnable Java SE records application that brings together language fundamentals, object-oriented design, exception handling, Streams, NIO.2, date/time APIs, recursion, nested types, enums, and recognized design patterns. Its strongest quality is the direct traceability between syllabus concepts and working application features. Completing the genuine screenshot evidence and adding automated tests would make the submission substantially stronger and easier to evaluate.

12. References

Reference	Use in project/report
ProgrammingInJavaProject1 brief (8 pages)	Functional, technical, package, demonstration, and submission requirements
Oracle Java SE Documentation - https://docs.oracle.com/en/java/	Java language and standard API reference
Eclipse IDE Help - https://help.eclipse.org/	IDE project and run-configuration terminology
GitHub repository - https://github.com/axbhijit4192-alt/CCRM-Java-Abhijit	Verified implementation source and documentation

Appendix A - Source inventory

Package	Classes/interfaces
cli	CCRMAApp
config	AppConfig
domain	Course, CourseCode, Enrollment, Grade, Instructor, Persistable, Person, ProfileLabel, RecordLabel, Searchable, Semester, Student, Transcript
exception	DuplicateEnrollmentException, MaxCreditLimitExceededException
io	BackupService, ImportExportService
service	CourseService, DataStore, EnrollmentService, StudentService, TranscriptService
util	ArrayDemos, FileUtils

Appendix B - Main menu

```

===== CCRM =====
1) Manage Students
2) Manage Courses
3) Enrollment & Grades
4) Import/Export
5) Backup & Show Size
6) Reports
7) Exit

```